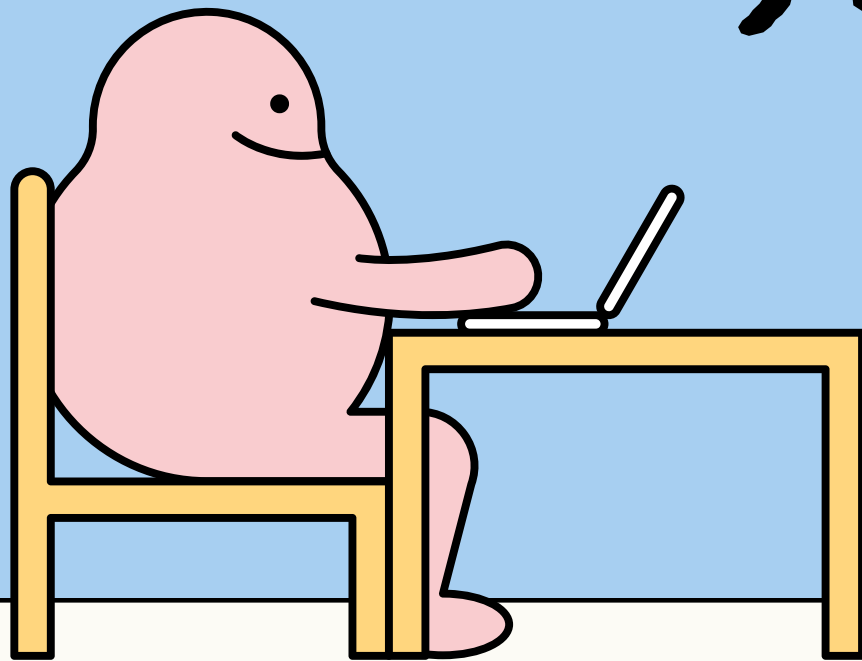


이번 주 수업 주제

SEMTLE 세미나

Jwt는 어떻게 인증의
강자가 되었는가...



오늘의 학습 목표는?

1

Jwt 이해하기

2

제(Jwt)어강

3

jwt 장단점

4

jwt vs Cookie/Session

Jwt는 어떻게 인증의 강자가 되었는가...



오늘 수업의 핵심 내용

- Json Web Token의 개념
어떻게 구현되어 있는지
- 장단점은 뭔지
Cookie & Session과 비교

사용자 인증 방식

3

1

전통 인증

2

Cookie /
Session

3

Web Token

4

2단계 인증

5

생체 인증

Jwt란?

Json Web Token

1 JWT는 “디지털 입장권”

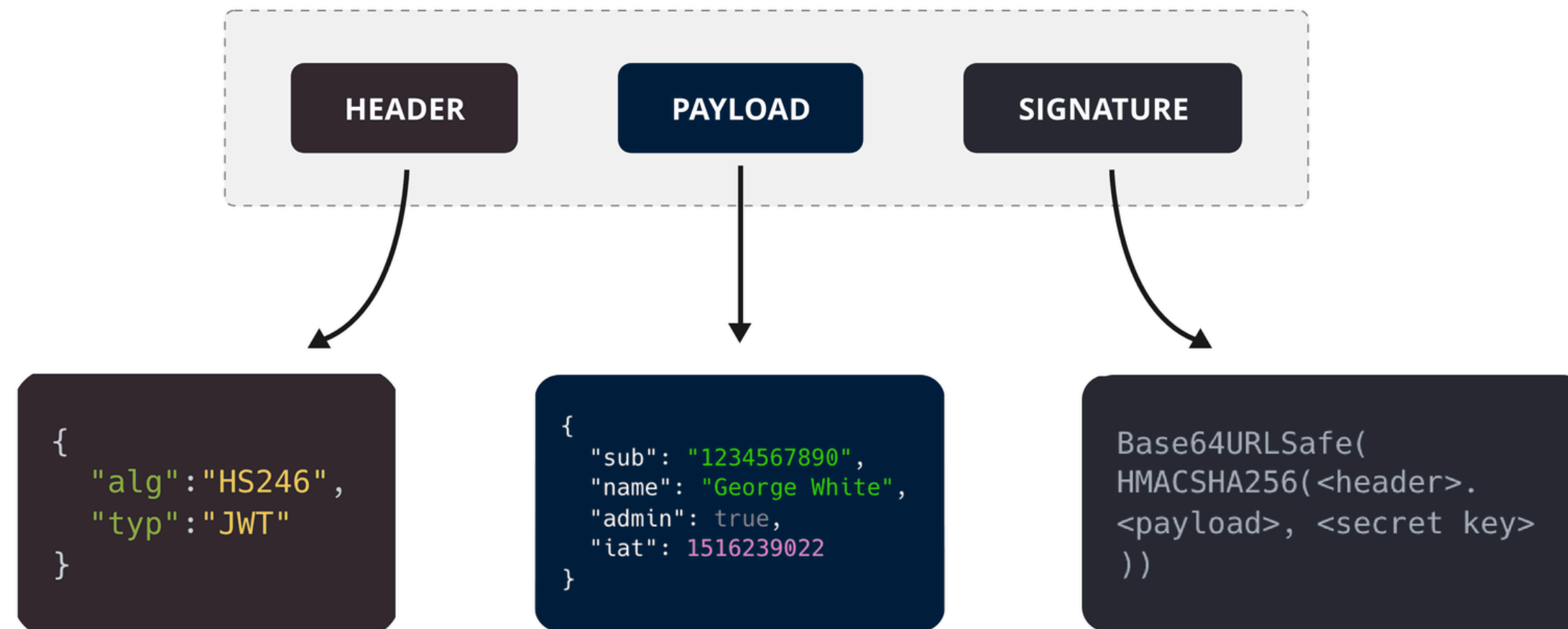
2 Header(헤더), Payload(내용), Signature(서명)

3 React, Flutter, Android 등 다양한 앱에서 인증 수단으로 사용

Structure of a JSON Web Token (JWT)



Structure of a JSON Web Token (JWT)



Jwt 구조

Header

- alg: 서명에 사용할 알고리즘 (예: HS256)
- typ: 토큰 타입 (보통 "JWT")

Payload

- sub: 사용자 식별자 (예: user ID)
- exp: 토큰 만료 시간
- iat: 토큰 발급 시간
- custom claims: 사용자 정보, 권한 등

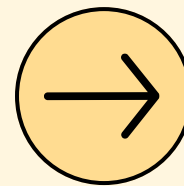
Signature

- 내용: 위조 방지를 위한 서명 값
- Header와 Payload를 조합한 서명

Jwt는 어떻게 인증의 강자가 되었는가...

Session 방식

- 서버가 사용자 세션을 직접 관리 → 메모리 부담
- 서버 확장 시 세션 동기화 필요
→ 로드 밸런싱 어려움
- 모바일, SPA, API 시대에 부적합한 구조



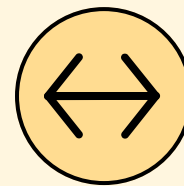
Jwt 방식

- 토큰 기반 인증으로 서버의 상태 저장 불필요
- 클라이언트가 인증 정보를 보관
→ 확장성과 유연성 향상
- OAuth2, 모바일 앱, SPA 등과 완벽한 궁합
- 위변조 방지 가능한 서명(Signature) 구조

Jwt는 어떻게 인증의 강자가 되었는가...

장점

- Stateless 구조
 - 서버에서 세션 상태 저장 필요 없음
 - 로드 밸런싱, 분산 시스템에 최적화
- 토큰 자체에 정보 포함
 - 빠른 인증 처리 가능 (DB 조회 최소화)
- 서명(Signature)으로 위·변조 방지
 - 클라이언트가 토큰을 변경해도 서버에서 탐지



단점

- 토큰 탈취 시 위험
 - 탈취되면 만료 시까지 유효
 - 짧은 만료시간, HTTPS 필수
- 토큰 무효화 어려움
 - 서버가 상태를 저장하지 않기 때문에 실시간 강제 로그아웃 어려움
- Payload는 인코딩일 뿐 암호화 아님
 - Base64로 누구나 복호화 가능
 - 민감정보 저장 X

Jwt vs Session/Cookie

그래서 뭐 써야 돼? - Session/Cookie

- 관리자 페이지, 사내 시스템
→ 내부망, 로그인 유지 중요, 실시간 로그아웃 필요
- 사용자 수 적은 서비스
→ 개인 블로그, 중소기업 웹사이트
- 네이버, 다음 등 웹사이트, 쇼핑몰 등

Jwt vs Session/Cookie

그래서 뭐 써야 돼? - Jwt

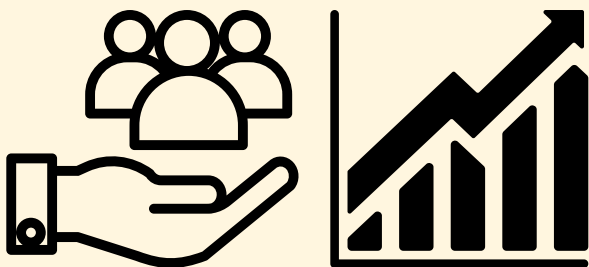
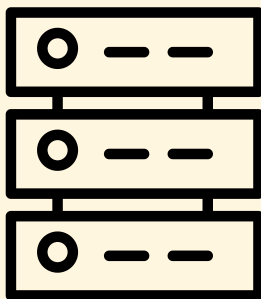
- 모바일 앱 / API 서버
→ 세션 공유 없이 인증 처리 가능
- 프론트엔드 분리형(SPA) 구조
→ 클라이언트가 토큰으로 API 요청
- 카카오톡, 넷플릭스, 인스타그램, 깃허브, 슬랙 등등

Session

그래서 뭐 !!!!

Jwt

WIN!



WIN! WIN!
WIN! WIN!
WIN! WIN!

Semtle 세미나 자료

수고했어요!

다음 시간에
만나요!

